

Урок 13. Корутины. Обработка исключений

13 мая 2020

В этом уроке рассмотрим как можно обработать исключения, возникающие в корутине. Разберем, что происходит под капотом и как работает `CoroutineExceptionHandler`.

Обработка исключений - это непростая тема. Начнем ее изучение с простых случаев, когда у нас есть `scope` и его корутины. На них мы рассмотрим механизмы обработки. А после этого уже перейдем к более сложным для понимания примерам с родительскими и дочерними корутинами.

Итак, у нас в методе `onRun` стартует простая корутина, в которой происходит ошибка:

```
1 scope = CoroutineScope(Job() + Dispatchers.Default)
2
3 private fun onRun() {
4
5     scope.launch {
6         Integer.parseInt("a")
7     }
8
9 }
```

Рассмотрим, какие у нас есть возможности для перехвата и обработки этой ошибки, чтобы приложение не завершило работу с крашем.

Но для начала рассмотрим решение, которое точно не сработает, хотя и кажется вполне рабочим. Важно понять, почему так происходит.

Обернуть корутину в try-catch

Обвернем весь вызов корутины в `try-catch`. И добавим логирование, чтобы лучше увидеть, что произойдет.

```
1 private fun onRun() {
2     log("onRun start")
3
4     try {
5         scope.launch {
6             Integer.parseInt("a")
7         }
8     } catch (e: Exception) {
9         log("error $e")
10    }
11
12    log("onRun end")
13 }
```

Интуитивно кажется, что все должно быть в порядке.

Запускаем, смотрим логи:

```
onRun start
onRun end
E/AndroidRuntime: FATAL EXCEPTION: DefaultDispatcher-worker-2
Process: com.startandroid.coroutinescourse, PID: 6718
java.lang.NumberFormatException: For input string: "a"
at java.lang.Integer.parseInt(Integer.java:615)
at java.lang.Integer.parseInt(Integer.java:650)
at com.startandroid.coroutinescourse.MainActivity$onRun$1.invokeSuspend(MainActivity.kt:59)
at kotlin.coroutines.jvm.internal.BaseContinuationImpl.resumeWith(ContinuationImpl.kt:33)
at kotlinx.coroutines.DispatchedTask.run(Dispatched.kt:241)
at kotlinx.coroutines.scheduling.CoroutineScheduler.runSafely(CoroutineScheduler.kt:594)
at kotlinx.coroutines.scheduling.CoroutineScheduler.access$runSafely(CoroutineScheduler.kt:60)
at kotlinx.coroutines.scheduling.CoroutineScheduler$Worker.run(CoroutineScheduler.kt:740)
```

Приложение свалилось с крашем. Давайте разбираться почему.

Вспомним похожие простые примеры, которые мы делали в прошлых уроках. В них мы выяснили, что метод `onRun` использует билдер `launch`, чтобы создать и запустить корутину, и сам после этого сразу завершается. А корутина живет своей жизнью в отдельном потоке. Вот именно поэтому `try-catch` здесь и не срабатывает.

Что делает билдер `launch`? Если вкратце, он формирует контекст, создает пару `Continuation+Job`, и отправляет `Continuation` диспетчеру, который помещает его в очередь. Ни в одном из этих шагов не было никакой ошибки, поэтому `try-catch` ничего не поймал. Билдер завершил свою работу, и метод `onRun` успешно завершился.

У диспетчера есть свободный поток, который постоянно мониторит очередь. Он обнаруживает там `Continuation` и начинает его выполнение. И вот тут уже возникает `NumberFormatException`. Но наш `try-catch` до него никак не мог дотянуться. Т.к. он покрывал только создание и запуск корутины, но не выполнение корутины, т.к. выполнение ушло в отдельный поток.

Обратите внимание на стек методов в логе. Читаем их снизу вверх.

CoroutineScheduler\$Worker - это поток, который из очереди берет задачи и выполняет их. Система выполняет его метод `run`.

DispatchedTask - обертка, в которой находится `Continuation`. В его методе `run` происходит вызов `Continuation.resumeWith`. Это и есть тот самый `resume` метод, про который я часто говорил в прошлых уроках про `Continuation` и диспетчеры. В этом методе происходит вызов метода `invokeSuspend`, в котором содержится наш код корутины:
`MainActivity$onRun$1.invokeSuspend`

Т.е. **MainActivity\$onRun\$1** - это `Continuation` класс. И если вы найдете его в папке `\app\build\tmp\kotlin-classes\debug\<package_name>` и декомпилируете, то сможете увидеть примерный код этого класса.

Метод **invokeSuspend** выполняет код корутины, и в нем то и происходит исключение, когда Kotlin пытается распарсить букву в цифру (`Integer.parseInt`).

Выполнение этого кода в потоке диспетчера мы не оборачивали в `try-catch`, поэтому исключение не поймали.

Можно привести аналогичный пример:

```
1 private fun onRun() {
2     log("onRun start")
3     try {
4         thread {
5             Integer.parseInt("a")
6         }
7     } catch (e: Exception) {
8         log("error $e")
9     }
10    log("onRun end")
11 }
```

Вместо корутины мы просто создаем новый поток. И в нем выполняем код, который выбросит исключение. Корутина работает именно по этому принципу, если рассматривать ее упрощенно.

Смотрим лог:

```
onRun start
onRun end
E/AndroidRuntime: FATAL EXCEPTION: Thread-2
Process: com.startandroid.coroutinescourse, PID: 6881
java.lang.NumberFormatException: For input string: "a"
at java.lang.Integer.parseInt(Integer.java:615)
at java.lang.Integer.parseInt(Integer.java:650)
at com.startandroid.coroutinescourse.MainActivity$onRun$1.invoke(MainActivity.kt:59)
at com.startandroid.coroutinescourse.MainActivity$onRun$1.invoke(MainActivity.kt:21)
at kotlin.concurrent.ThreadsKt$thread$thread$1.run(Thread.kt:30)
```

Похоже на предыдущий случай. Система выполнила у потока метод `run`. А тот выполнил код внутри блока `thread`, получил исключение и крашнул приложение.

А наш `try-catch` снова не у дел. Потому что он покрывал создание и запуск нового потока, но не выполнение кода в нем.

Многовато получилось объяснений для способа, который не ловит исключения. Но, я надеюсь, они помогли вам лучше понять работу корутин под капотом и почему, казалось бы, очевидный способ не работает.

Теперь давайте рассмотрим пару способов, как справиться с ошибкой и избежать краша.

Обернуть ошибку в try-catch

Первый способ - самый очевидный и простой: поместить в `try-catch` код, который выбрасывает ошибку. В этом случае все работает, как и должно.

```
1 scope.launch {
2     try {
3         Integer.parseInt("a")
4     } catch (e: Exception) {
5         log("exception $e")
6     }
7 }
```

Корутина успешно выполнит свой код и завершится. Краша не будет. А в логах мы увидим:

```
exception java.lang.NumberFormatException: For input string: "a"
```

CoroutineExceptionHandler

Второй способ - это обработчик ошибок, который мы сами можем создать и поместить в контекст корутины. Чтобы лучше понять, как корутина использует этот обработчик, снова покопаемся в ее внутренностях.

Когда в корутине происходит исключение, которое не обернуто в `try-catch`, мы получаем краш приложения. В этом мы убедились чуть ранее в этом уроке. Но на самом деле под капотом происходила определенная работа по обработке этого исключения. Оно было перехвачено, но потом снова выброшено. Разберем этот момент подробнее.

Как мы уже знаем, в Continuation метод resumeWith вызывает invokeSuspend. Этот вызов обернут в try-catch. И если код в invokeSuspend (т.е. наш код корутины) выбросит исключение, то оно будет перехвачено и сохранено, как результат работы корутины. Continuation передает этот результат в Job. Тем самым он сообщает, что завершил выполнение своего кода.

Job проверяет, что ему пришло на вход. Если бы ему пришла не ошибка, а реальный результат (или Unit), то Job поменял бы статус корутины (т.е. самого себя) в Завершена и корутина считалась бы успешно завершенной. Но в нашем случае пришла ошибка, и вместо простого завершения Job выполняет определенный набор действий.

Он сообщает об ошибке родителю. Родитель в нашем случае - это Job в scope. Узнав о том, что в его дочерней корутине произошла ошибка, он отменяет себя, а также все свои дочерние корутины. Т.е. - все корутины, созданные в этом scope. Чуть позже мы на примере увидим, как это происходит. Для взаимодействия между собой корутины используют связи, которые мы проходили в прошлом уроке.

Далее Job корутины пытается самостоятельно обработать исключение, которое он получил из Continuation. Для этого он проверяет свой контекст, есть ли там объект CoroutineExceptionHandler.

Если Job находит этот объект, он передает ему исключение и крэша не будет. Если же такого объекта в контексте не было, то Job отправит исключение в глобальный обработчик ошибок, который завершит работу приложения с крэшем.

Проговорим то же самое еще раз, но покороче. Job получает из Continuation ошибку. Он сообщает об этом родителю (scope), который по такому поводу отменяет себя и всех своих детей. А сам Job пытается передать ошибку в CoroutineExceptionHandler. Если такого обработчика ему не предоставили, то ошибка уходит в глобальный обработчик, что приводит к крэшу приложения.

Давайте рассмотрим пример:

```
1 val handler = CoroutineExceptionHandler { context, exception ->
2     log("handled $exception")
3 }
4
5 scope.launch(handler) {
6     Integer.parseInt("a")
7 }
```

Создаем обработчик CoroutineExceptionHandler, который просто пишет в лог ошибку, и передаем его в билдер корутины, чтобы он попал в ее контекст.

Выполнив этот код мы в логах увидим, что исключение было обработано нашим объектом CoroutineExceptionHandler:

handled java.lang.NumberFormatException: For input string: "a"

Корутина отправила исключение в наш обработчик, вместо глобального, и, тем самым, мы избежали крэша.

Теперь давайте рассмотрим нюансы этого способа обработки исключений.

Отмена корутин

Даже несмотря на обработчик ошибок, корутина, в которой произошло исключение, сообщит об ошибке в родительский scope, а тот отменит все свои дочерние корутины.

Убедимся в этом на простом примере:

```
1 val handler = CoroutineExceptionHandler { context, exception ->
2     log("first coroutine exception $exception")
3 }
4
5 scope.launch(handler) {
6     TimeUnit.MILLISECONDS.sleep(1000)
7     Integer.parseInt("a")
8 }
9
10 scope.launch {
11     repeat(5) {
12         TimeUnit.MILLISECONDS.sleep(300)
13         log("second coroutine isActive ${isActive}")
14     }
15 }
```

Запускаем две корутины в одном scope. Первая корутина вызовет исключение через 1000 мсек. Вторая корутина 5 раз с интервалом в 300 мсек выводит в лог свой статус.

Лог:

```
15:36:34.043 second coroutine isActive true
15:36:34.344 second coroutine isActive true
15:36:34.646 second coroutine isActive true
15:36:34.745 first coroutine exception java.lang.NumberFormatException: For input string: "a"
15:36:34.947 second coroutine isActive false
15:36:35.248 second coroutine isActive false
```

Поначалу вторая корутина сообщает о том, что полет нормальный и все ок. Далее в первой корутине происходит исключение и уходит в обработчик, который пишет об этом в лог. А scope, узнав, что в первой корутине произошла ошибка, отменяет вторую. Это видно по ее статусу в логе.

Это может быть удобным, если вам при возникновении ошибки в одной операции, надо отменить другие операции. Вы просто помещаете корутины с этими операциями в один scope.

Учитывайте, что scope отменяет не только корутины, но и себя. А это означает, что в этом scope мы больше не сможем запустить корутины.

Разные scope никак не связаны между собой. Давайте это проверим на том же самом примере, но поместим первую и вторую корутину в разные scope.

```
1 | val handler = CoroutineExceptionHandler { context, exception ->
2 |     log("first coroutine exception $exception")
3 | }
4 |
5 | scope.launch(handler) {
6 |     TimeUnit.MILLISECONDS.sleep(1000)
7 |     Integer.parseInt("a")
8 | }
9 |
10 | scope2.launch {
11 |     repeat(5) {
12 |         TimeUnit.MILLISECONDS.sleep(300)
13 |         log("second coroutine isActive ${isActive}")
14 |     }
15 | }
```

Вторая корутина теперь вызывается в другом scope.

Лог:

```
15:37:32.830 second coroutine isActive true
15:37:33.132 second coroutine isActive true
15:37:33.435 second coroutine isActive true
15:37:33.535 first coroutine exception java.lang.NumberFormatException: For input string: "a"
15:37:33.736 second coroutine isActive true
15:37:34.038 second coroutine isActive true
```

Ошибка в корутине в первом scope никак не повлияла на корутину во втором.

Не забывайте, что контекст корутин формируется не только из того, что было передано в билдер, но и того, что пришло из контекста родителя. Поэтому, если вам надо поместить ваш CoroutineExceptionHandler обработчик во все корутины, то просто поместите его в scope, и он будет передан во все корутины, созданные в нем.

```
1 | val handler = CoroutineExceptionHandler { context, exception ->
2 |     log("coroutine exception $exception")
3 | }
4 |
5 | val scope = CoroutineScope(Job() + Dispatchers.Default + handler)
```

SupervisorJob

В примерах с CoroutineExceptionHandler мы убедились, что scope отменяет своих детей, когда в одном из них происходит ошибка. Пусть даже эта ошибка и была передана в обработчик. Такое поведение родителя далеко не всегда может быть удобным. Поэтому у нас есть возможность это отключить.

Для этого надо в scope вместо обычного Job() использовать SupervisorJob(). Он отличается от Job() тем, что не отменяет всех своих детей при возникновении ошибки в одном из них.

Рассмотрим снова пример с двумя корутинами одного scope, но теперь в scope в качестве Job используем SupervisorJob()

```
1 | val handler = CoroutineExceptionHandler { context, exception ->
2 |     log("first coroutine exception $exception")
3 | }
4 |
5 | val scope = CoroutineScope(SupervisorJob() + Dispatchers.Default + handler)
6 |
7 | scope.launch {
8 |     TimeUnit.MILLISECONDS.sleep(1000)
9 |     Integer.parseInt("a")
10 | }
11 |
12 | scope.launch {
13 |     repeat(5) {
14 |         TimeUnit.MILLISECONDS.sleep(300)
15 |         log("second coroutine isActive ${isActive}")
16 |     }
17 | }
```

Запускаем, смотрим лог:

```
14:34:36.641 second coroutine isActive true
14:34:36.943 second coroutine isActive true
14:34:37.244 second coroutine isActive true
14:34:37.280 first coroutine exception java.lang.NumberFormatException: For input string: "a"
14:34:37.546 second coroutine isActive true
14:34:37.955 second coroutine isActive true
```

После того, как первая корутина выбросила исключение, score не стал отменять вторую корутину.

На этом пока закончим. В следующем уроке рассмотрим, как это все будет работать в случае с вложенными корутинами.